

Introduction to Computer Programming

Prof. Alessio Martino

amartino@luiss.it

Department of Business and Management
Viale Romania 32, 00197 Rome



A.Y. 2024-2025

Outline

- 1 Stacks
 - Simulating a Stack
- 2 Queues
 - Simulating a Queue
 - Queue vs. Stack
- 3 Recursion
 - Recursion and Problem Solving
 - Examples
 - Recursion and Problem Solving (pt.2)
- Issues with Recursion
- 4 Memory Management
 - Case Study: `countdown`
- 5 Exercises
 - `factorial`
 - `palindrome`
 - `fibonacci`
- 6 References
- 7 Acknowledgments

Stacks

The **stack** is one of the most popular abstract data types in computer science.

A stack is a sequence (collection) of elements where only the "last" element (i.e., the one at the "top" of the stack) can be accessed.

The allowed operations on a stack $\mathcal{S}$ are the following:

- **push**($e, \mathcal{S}$): adds a new element $e \in \mathcal{V}$, for any domain $\mathcal{V}$, at the top of the stack $\mathcal{S}$
- **pop**($\mathcal{S}$): returns and removes the current element at the top of the stack $\mathcal{S}$.

Optionally, it is possible to define an additional operation:

- **peek**($\mathcal{S}$): reads the top element without modifying the stack.

A special domain value is $\mathcal{S}_\emptyset$, namely the empty stack.

As $\mathcal{S}_\emptyset$ has no elements:

- `pop()` is not allowed
- `peek()` is not allowed.

The stack is a LIFO (Last-In, First-Out) data structure, because:

- the last element that entered the stack (i.e., via `push()`)
- ... is the one that will be removed first (i.e., via `pop()`)

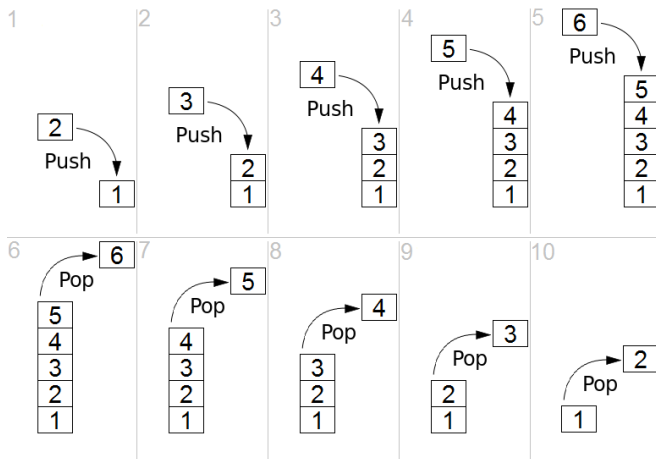


Figure 1: push() and pop() over a stack.

You can simulate the behaviour of a stack with lists.

```
S = [ ]           # start with an empty stack  $\mathcal{S}_0$ 
S.insert(0, 1)
S.insert(0, 2)    # step 1 in previous slide
S.insert(0, 3)    # step 2 in previous slide
S.insert(0, 4)    # step 3 in previous slide
S.insert(0, 5)    # step 4 in previous slide
S.insert(0, 6)    # step 5 in previous slide
S.pop(0)          # step 6 in previous slide
S.pop(0)          # step 7 in previous slide
S.pop(0)          # step 8 in previous slide
S.pop(0)          # step 9 in previous slide
S.pop(0)          # step 10 in previous slide
```

Recall

`lst.insert(x, y)` inserts value `y` at position `x` in list `lst`.

Note

Previously, we supposed that the head of the list is the top of the stack. You can also assume that the tail of the list is the head of the stack.

```
S = [ ]           # start with an empty stack  $S_0$ 
S.append(1)
S.append(2)       # step 1 in slide 4
S.append(3)       # step 2 in slide 4
S.append(4)       # step 3 in slide 4
S.append(5)       # step 4 in slide 4
S.append(6)       # step 5 in slide 4
S.pop()           # step 6 in slide 4
S.pop()           # step 7 in slide 4
S.pop()           # step 8 in slide 4
S.pop()           # step 9 in slide 4
S.pop()           # step 10 in slide 4
```

Queues

The **queue** is another popular abstract data type in computer science.

A queue is a sequence (collection) of elements where only the "oldest" element can be accessed.

The allowed operations on a queue Q are the following:

- **enqueue**(e, Q): adds a new element $e \in \mathcal{V}$, for any domain $\mathcal{V}$, at the beginning (back) of the queue Q
- **dequeue**(Q): returns and removes the current element at the end (front) of the queue Q .

Optionally, it is possible to define an additional operation:

- **peek**(Q): reads the last element without modifying the queue.

A special domain value is $Q_\emptyset$, namely the empty queue.

As $Q_\emptyset$ has no elements:

- `dequeue()` is not allowed
- `peek()` is not allowed.

The queue is a FIFO (First-In, First-Out) data structure, because:

- the first element that entered the queue (i.e., via `enqueue()`)
- ... is the one that will be removed first (i.e., via `dequeue()`)

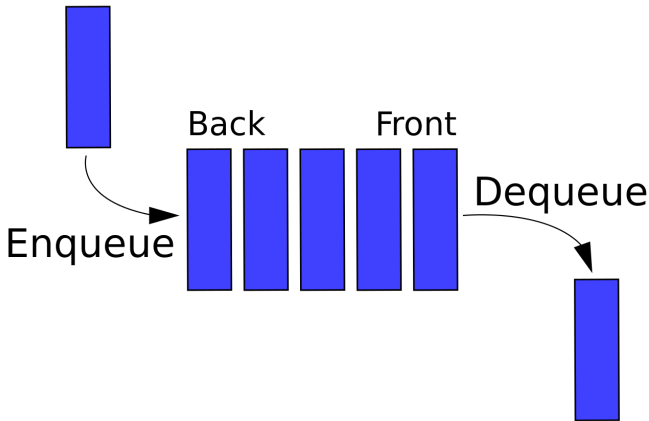


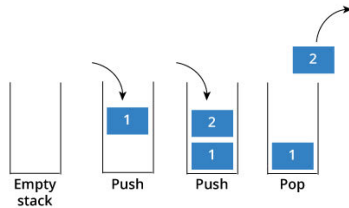
Figure 2: Queues

You can simulate the behaviour of a queue with lists.

```
Q = []          # start with an empty queue  $Q_0$ 
Q.insert(0, 1)  # insert at the beginning = enqueue
Q.insert(0, 2)
Q.insert(0, 3)
Q.insert(0, 4)
Q.insert(0, 5)
Q.insert(0, 6)
Q.pop()         # pop at the end = dequeue
Q.pop()
Q.pop()
Q.pop()
Q.pop()
```

Note

Here we suppose that the head of the list is the beginning (back) of the queue and that the tail of the list is the end (front) of the queue.



(a) Stack



(b) Queue

Figure 3: Queue vs. Stack (serious version)



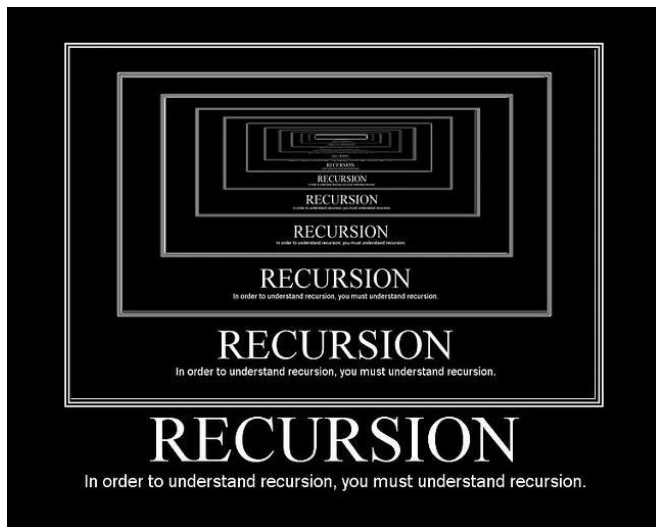
(a) Stack



(b) Queue

Figure 4: Queue vs. Stack (not-so-serious version)

Recursion



When designing the solution of a problem

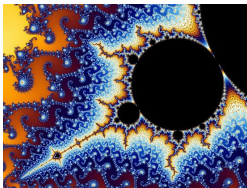
... try to solve the problem assuming that you have *already solved it!*

How is that possible?

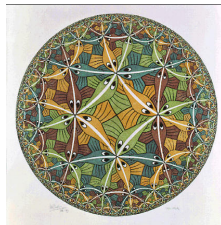
Define the solution of a problem instance in terms of the solution of *smaller* problem instances.

Practically speaking:

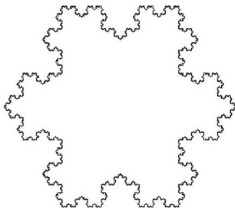
The function that computes the solution will *invoke itself* (once or more than once) from inside its body.



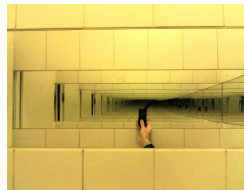
(a) Mandelbrot Set



(b) Escher's *Circle Limit III*



(c) Koch Snowflake



(d) Parallel Mirrors

Figure 5: Real-world recursion

A more practical example? Computing the factorial of a number n .

$$n! = n \cdot (n - 1) \cdot (n - 2) \cdot \dots \cdot 1$$

Back-of-the-envelope calculation: $5!$

$$\begin{array}{c} 5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot \underbrace{1}_{1!} \\ \underbrace{\hspace{1.5cm}}_{2 \cdot 1!} \\ \underbrace{\hspace{1.5cm}}_{3 \cdot 2!} \\ \underbrace{\hspace{1.5cm}}_{4 \cdot 3!} \\ \underbrace{\hspace{1.5cm}}_{5 \cdot 4!} \end{array}$$

We can spot the following pattern:

$$n! = n \cdot (n - 1)!$$

with base case

$$0! = 1$$

In order to use a recursive approach:

- 1 you must be able to **decompose** the problem into problems of the same type
- 2 you must find a **relation** that connects a given problem to the solution of similar subproblems
- 3 you need to know the solution of a special case of the problem (the **base case**, where the recursion terminates).

Take the factorial example:

- 1 multiplication between two numbers
- 2 multiplication between n and the already-known $(n - 1)!$
- 3 the base case is $0! = 1$

There are two major issues with recursion:

① **Infinite recursion**

- missing base case?
- function called with wrong arguments?

For example, what happens if we do $(-1)!$?

... more on this later

② **Un-responsiveness**

- your code can easily become very slow, also with "easy" input parameters
- ... more on memory management later

Note

Python is quite good at catching infinite recursions: it has a dedicated error.

RecursionError: maximum recursion depth exceeded in comparison

Memory Management

Write a recursive function `countdown(n)` that takes a positive integer `n` and prints the countdown. As the countdown reaches 0, prints **Blastoff!** and quits.

```
>>> def countdown(n):  
>>>     if n==0:  
>>>         print("Blastoff!")  
>>>     else:  
>>>         print(n)  
>>>         countdown(n-1)
```

```
>>> countdown(3)  
3  
2  
1  
Blastoff!
```

What's going on here?

- ① The execution of `countdown` begins with `n=3`, and since `n` is greater than 0, it outputs the value 3, and then calls itself ...
- ② The execution of `countdown` begins with `n=2`, and since `n` is greater than 0, it outputs the value 2, and then calls itself ...
 - ③ The execution of `countdown` begins with `n=1`, and since `n` is greater than 0, it outputs the value 1, and then calls itself ...
 - ④ The execution of `countdown` begins with `n=0`, and since `n` is not greater than 0, it outputs **Blastoff!** and then returns.
 - ⑤ The countdown that got `n=1` returns.
- ⑥ The countdown that got `n=2` returns.
- ⑦ The countdown that got `n=3` returns.

And we are back into the `main` script.

How does the stack in Figure 6 works?

- ① (as usual) at the top of the stack is the frame for the **main**¹
- ② we call **countdown(3)**, so there is a new frame in the stack
- ③ **countdown(3)** calls **countdown(2)**, so there is a new frame in the stack
- ④ **countdown(2)** calls **countdown(1)**, so there is a new frame in the stack
- ⑤ **countdown(1)** calls **countdown(0)**, so there is a new frame in the stack
- ⑥ **countdown(0)** does not make any further calls, so no more frames

We did a bunch of **pushes**, now we must do some **pops**:

- ① **countdown(0)** finishes its execution (i.e., prints **Blastoff!** without doing anything else). Its frame is erased.
- ② the control goes back to **countdown(1)**, which finishes its execution. Its frame is erased.
- ③ the control goes back to **countdown(2)**, which finishes its execution. Its frame is erased.
- ④ the control goes back to **countdown(3)**, which finishes its execution. Its frame is erased.
- ⑤ the control goes back to the **main**.

¹In our case, it is empty because we did not create any variables in **main** or pass any arguments to it.

Exercises

Write a recursive function `factorial(n)` that takes as input a positive integer `n` and returns `n!`.

```
>>> def factorial(n):  
>>>     if n==0:  
>>>         return 1  
>>>     else:  
>>>         return n * factorial(n-1)
```

```
>>> output = factorial(5)  
>>> print(output)  
120
```

Warning

What happens when calling `factorial(-1)`?

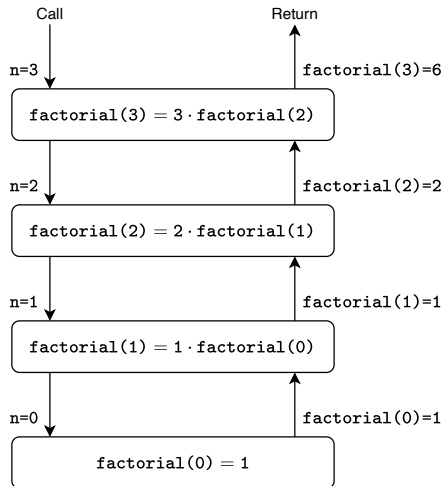


Figure 7: Stack diagram for `factorial(3)`.

Write a recursive function `palindrome(s)` that takes as input a string `s` and returns `True` if `s` is palindrome² and `False` otherwise.

Decomposition strategy: let `s` be "racecar"

racecar	same letter ✓
aceca	same letter ✓
cec	same letter ✓
e	trivial ✓

and now let `s` be "rediviuer"

rediviuer	same letter ✓
ediviu	same letter ✓
diviu	different letter ✗

Hence, "racecar" is palindrome, but "rediviuer" it's not.

²A word (or any other sequence of characters) is said to be *palindrome* if it reads the same backward as forward.

```
>>> def palindrome(s):
>>>     if len(s) <= 1:
>>>         return True
>>>     else:
>>>         return (s[0] == s[-1]) and palindrome(s[1:-1])

>>> output1 = palindrome("racer")
>>> print(output1)
True
>>> output2 = palindrome("rediviuer")
>>> print(output2)
False
```

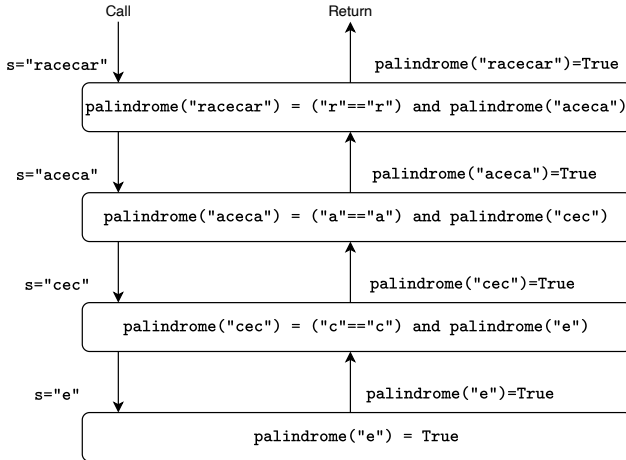


Figure 8: Stack diagram for `palindrome("racecar")`.

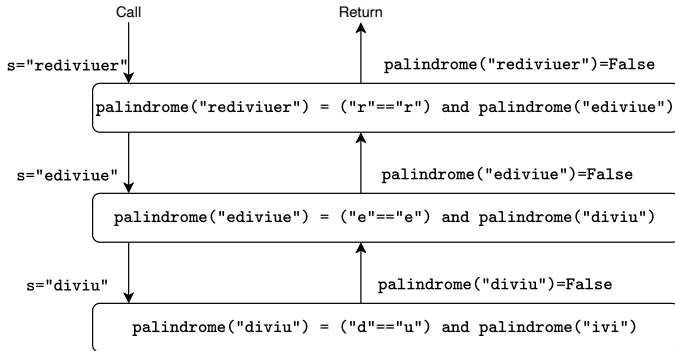


Figure 9: Stack diagram for `palindrome("rediviuer")`.

Write a recursive function `fibonacci(n)` that takes as input a positive integer `n` and returns the n^{th} Fibonacci number.

Recall

Each number in the Fibonacci sequence is given by the sum of the two preceding ones:

0 1 1 2 3 5 8 11 ...

Decomposition strategy: `fibonacci(n)` with $n=5^3$.

$$\begin{array}{c} \underbrace{}_5 \\ \underbrace{}_3 + \underbrace{}_2 \\ \underbrace{}_2 + 1 \quad 1 + 1 \\ \underbrace{}_2 \\ 1 + 1 \end{array}$$

³By coincidence, the 5th Fibonacci number is 5.

```
>>> def fibonacci(n):  
>>>     if n in [0, 1]:  
>>>         return n  
>>>     else:  
>>>         return (fibonacci(n-1) + fibonacci(n-2))  
  
>>> output = fibonacci(5)  
>>> print(output)  
5
```

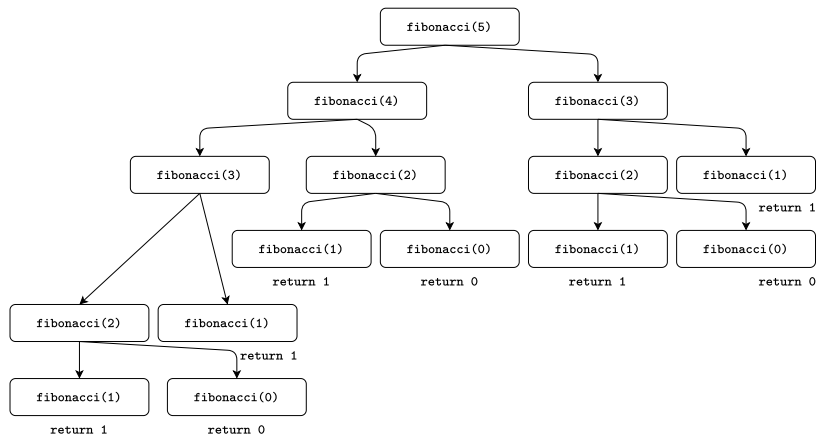


Figure 10: Call Stack diagram for `fibonacci(5)`.

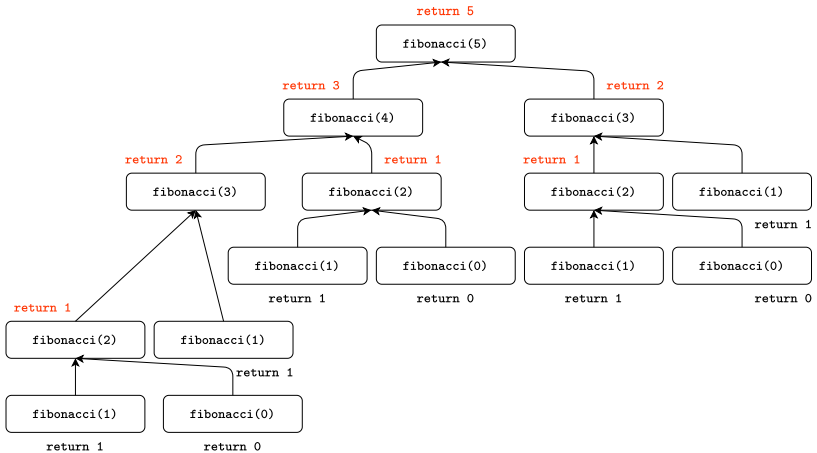


Figure 11: Return Stack diagram for `fibonacci(5)`.

References

- Guttag, J. V. (2013). *Introduction to Computation and Programming using Python (Revised and Expanded Edition)*. MIT Press. [Section 4.3]
- Downey, A. (2015). *Think Python*. Green Tea Press ed. 2nd. [Sections 5.8–5.10]

Acknowledgments

- [https://en.wikipedia.org/wiki/Stack_\(abstract_data_type\)](https://en.wikipedia.org/wiki/Stack_(abstract_data_type))
- [https://en.wikipedia.org/wiki/Queue_\(abstract_data_type\)](https://en.wikipedia.org/wiki/Queue_(abstract_data_type))
- <https://www.walesonline.co.uk/news/wales-news/swansea-roads-cars-traffic-bikes-21148566>